

Lecture 20: Concurrency of Indexes

B+ Tree Optimizations and Concurrency Control

Amit Kumar Dhar

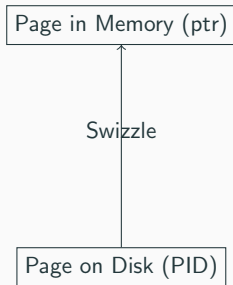
November 6, 2025

Optimizations on B+ Tree Data Structures

Pointer Swizzling

Definition: Pointer swizzling is the process of converting pointers from an on-disk format to an in-memory format. **Explanation:**

When a page is read from disk into memory, the pointers to other pages are "swizzled" to be direct memory pointers. This avoids the overhead of looking up the page in the buffer pool every time a pointer is traversed.



Pointer Swizzling: Advantages

- **Speed:** Direct memory pointers are much faster to traverse than looking up a page ID in the buffer pool.
- **Simplicity:** Code can be written as if the data is entirely in memory.

Other Optimization: Modifying is Expensive

- **Why is it expensive?**

- Each modification to a B+ tree node requires a write to disk.
- Disk I/O is slow.
- If a node is modified frequently, it can lead to a lot of disk I/O.

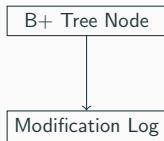
- **How to optimize?**

- Delay updates to nodes.
- Batch updates together.

Write Optimized B+tree

- **Basic Idea:** Delay updates to B+ tree nodes.
- **How it works:**
 - When a key is inserted or deleted, the change is written to a log.
 - The B+ tree is not updated immediately.
 - The log is periodically merged with the B+ tree.

Add Mod Log to B+tree Nodes



Index Concurrency Control

- **Observation:** So far, we have assumed a single thread of execution.
- Modern DBMSs run multiple threads concurrently.
- We need to make sure that the index is thread-safe.

Concurrency Control

- **Definition:** Concurrency control is the process of managing simultaneous access to shared resources.
- **Types:**
 - **Logical Correctness:** The concurrent execution of transactions must be equivalent to some serial execution of those transactions.
 - **Physical Correctness:** The internal data structures of the DBMS must be consistent.

Locks and Latches

- **Locks:**

- Protect logical contents of the database.
- Held for the duration of a transaction.
- Managed by the transaction manager.

- **Latches:**

- Protect critical sections of the DBMS's internal data structures.
- Held for a very short duration.
- Managed by the thread executing the operation.

- **Read Mode:**

- Multiple threads can hold a read latch on the same object.
- A thread can acquire a read latch if no other thread holds a write latch on the object.

- **Write Mode:**

- Only one thread can hold a write latch on an object.
- A thread can acquire a write latch if no other thread holds a read or write latch on the object.

Latch Goals

- Small memory footprint.
- Fast to execute.
- Decentralized management.
- Avoid system calls.

Test-and-Set Spin Latch

- **Advantages:**

- Very fast if the latch is not contended.
- No system calls.

- **Disadvantages:**

- Wastes CPU cycles if the latch is contended.
- Not fair.

Test-and-Set Spin Latch: C++ Implementation

```
class SpinLatch {  
public:  
    SpinLatch() : flag_(false) {}  
  
    void lock() {  
        while (flag_.exchange(true, std::memory_order_acquire)  
        }  
  
    void unlock() {  
        flag_.store(false, std::memory_order_release);  
    }  
  
private:  
    std::atomic<bool> flag_;  
};
```

Test-and-Set Spin Latch: C Implementation

```
#include <stdatomic.h>

typedef struct {
    atomic_flag flag;
} spinlock_t;

void spinlock_init(spinlock_t *lock) {
    atomic_flag_clear(&lock->flag);
}

void spinlock_lock(spinlock_t *lock) {
    while (atomic_flag_test_and_set(&lock->flag));
}

void spinlock_unlock(spinlock_t *lock) {
```

- **Advantages:**

- Does not waste CPU cycles if the latch is contended.
- Fair.

- **Disadvantages:**

- Slower than a spin latch if the latch is not contended.
- Requires system calls.

Blocking OS Mutex: C++ Implementation

```
#include <mutex>

class Mutex {
public:
    void lock() {
        mutex_.lock();
    }

    void unlock() {
        mutex_.unlock();
    }

private:
    std::mutex mutex_;
};
```

Blocking OS Mutex: C Implementation

```
#include <pthread.h>
typedef struct {
    pthread_mutex_t mutex;
} mutex_t;
void mutex_init(mutex_t *lock) {
    pthread_mutex_init(&lock->mutex, NULL);
}
void mutex_lock(mutex_t *lock) {
    pthread_mutex_lock(&lock->mutex);
}
void mutex_unlock(mutex_t *lock) {
    pthread_mutex_unlock(&lock->mutex);
}
```

- **Advantages:**

- Allows multiple readers to access the data structure concurrently.
- Can improve performance for read-heavy workloads.

- **Disadvantages:**

- More complex to implement than a simple mutex.
- Can lead to writer starvation.

Reader-Writer Latches: C++ Implementation

```
#include <shared_mutex>
class RWMutex {
public:
    void rlock() {
        mutex_.lock_shared();
    }
    void wlock() {
        mutex_.lock();
    }
    void unlock() {
        mutex_.unlock();
    }
private:
    std::shared_mutex mutex_;
};
```

Reader-Writer Latches: C Implementation

```
#include <pthread.h>

typedef struct {
    pthread_rwlock_t rwlock;
} rwlock_t;

void rwlock_init(rwlock_t *lock) {
    pthread_rwlock_init(&lock->rwlock, NULL);
}

void rwlock_rdlock(rwlock_t *lock) {
    pthread_rwlock_rdlock(&lock->rwlock);
}

void rwlock_wrlock(rwlock_t *lock) {
    pthread_rwlock_wrlock(&lock->rwlock);
}

void rwlock_unlock(rwlock_t *lock) {
    pthread_rwlock_unlock(&lock->rwlock);
}
```

Thank You
Questions?